

面向 LLM / Agent Systems 科研的深度学习零基础课

第 06 讲

Batch 与批量矩阵运算：从 (B, D) 走向 LLM 的 (B, T, D)

本讲目标：理解“很多样本/很多 Token 如何共享同一组参数并行计算”。重点掌握 Batch、Mini-batch、Batch Dimension、 (B, D) 与 (B, T, D) 、Broadcasting，以及 Linear 为什么可以直接作用于三维 Tensor 的最后一维。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

1 为什么这一讲是第一阶段的收口课	3
1.1 本讲最低验收标准	3
2 先把几个词彻底分开：Sample、Feature、Batch	3
2.1 Sample：一条数据	4
2.2 Batch：一次一起送进模型的一组样本	4
3 为什么要 Batch？	4
4 二维 Batch： $(B, D) @ (D, O) \rightarrow (B, O)$	5
4.1 每一行究竟发生了什么？	5
5 从 (B, D) 进入 LLM：为什么多出一个 T？	6
6 关键问题：Linear 怎样作用于三维 Tensor？	6
6.1 内部到底发生了什么？	6
7 一个极其重要的判断：Linear 本身不会混合 Token	7
8 为什么三维 Linear 等价于先展平 $B \times T$？	8
9 Bias 在三维 Tensor 中如何 Broadcasting？	8
10 Batch Size、Sequence Length 与参数量的关系	9
10.1 为什么 B 和 T 大了会更贵？	9
11 Mini-batch：真实训练中常用的折中	9
12 NumPy：直接验证 $(B, T, D) \rightarrow (B, T, O)$	10
12.1 怎样读这一段代码？	10
13 PyTorch：nn.Linear 为什么能直接处理三维 Tensor？	10
14 与 Transformer 的第一次完整 Shape 对接	11
14.1 FFN / MLP 也是同样的 Shape 思路	11
15 与 Agent Systems 有什么关系？	12
16 常见误区	12

17 本讲实验：从二维 Batch 到三维 Token Batch	13
18 本讲知识地图：第一阶段到这里闭环	13
19 练习题	14
20 答案	14
21 下一讲预告：从“线性计算”进入“神经元”	15

1 为什么这一讲是第一阶段的收口课

前五讲我们已经建立了：标量、向量、矩阵、Tensor、Shape、Dot Product、Matrix Multiplication，以及 $Y = XW + b$ 。但真实深度学习几乎不会“每次只处理一个孤立向量”。

训练时，我们通常一次处理很多样本；在 LLM 中，一个样本里又包含很多 Token。因此你以后最常见的输入不是 $(D,)$ ，而是：

$$(B, D) \text{ 或 } (B, T, D)$$

本讲最重要的一句话

Batch 并不是给每个样本复制一套模型参数，而是让很多样本（以及很多 Token）共享同一套 W, b ，一次并行完成相同类型的计算。

与 LLM / Transformer / Agent Systems 的连接

你以前在 LLM 课程里经常见过 hidden states 的 Shape 类似：

$$(B, T, D) = (\text{batch size}, \text{sequence length}, \text{hidden size})$$

以前可能只是“见过”。本讲要回答：为什么一个 `nn.Linear(D, O)` 可以直接吃进去三维 Tensor，并输出 (B, T, O) ？它到底对哪一维做了什么？

1.1 本讲最低验收标准

学完后你应该能做到：

1. 解释 Sample、Batch、Batch Size、Batch Dimension；
2. 看懂 (B, D) 与 (B, T, D) ；
3. 判断 $X : (B, D)$ 与 $W : (D, O)$ 相乘后的 Shape；
4. 解释 Batch 中所有样本为什么共享同一套参数；
5. 解释 (B, T, D) 经过 Linear 后为什么是 (B, T, O) ；
6. 说明 Linear 不会因为存在 T 轴就自动“混合 Token”；
7. 理解 Bias 在 (B, T, O) 上怎样 Broadcasting；
8. 解释为什么 Batch Size / Sequence Length 会影响计算量和显存，却不直接改变 Linear 的参数量；
9. 用 NumPy 与 PyTorch 验证三维 Linear；
10. 能把 Transformer 中 Q/K/V 投影的 Shape 写出来。

2 先把几个词彻底分开：Sample、Feature、Batch

2.1 Sample：一条数据

假设每个学生用 4 个数字表示：

[身高, 体重, 学习时长, 睡眠时长]

一个学生就是一个 **Sample（样本）**。

如果每个样本有 4 个数字，可以写：

$$x : (4,)$$

这里的 4 是 **Feature Dimension（特征维度）**。

2.2 Batch：一次一起送进模型的一组样本

如果一次取 3 个学生：

$$X = \begin{bmatrix} \text{样本 1} & \text{样本 2} & \text{样本 3} \end{bmatrix}$$

那么：

$$X : (3, 4)$$

可以记成：

$$X : (B, D)$$

其中：

- $B = \text{Batch Size}$ ：一次处理多少个样本；
- $D = \text{Feature Dimension}$ ：每个样本有多少个特征。

具体数字例子

若 $X.\text{shape} = (32, 768)$ ：

- 32 通常表示一次处理 32 个样本；
- 768 表示每个样本当前用 768 个数字表示。

不能机械地说“第一维永远是 Batch”。你仍然要看上下文，但深度学习里这是非常常见的约定。

3 为什么要 Batch？

最直观的原因是计算效率。现代 CPU/GPU 非常擅长把大量相同形式的数值运算一起做。

假设你有 1000 个样本。最笨的方法是：

1. 取第 1 个样本；
2. 做 $xW + b$ ；
3. 再取第 2 个；

4. 再做一次；
5. 重复 1000 次。

而 Batch 的思想是把很多样本堆起来：

$$X : (B, D)$$

然后一次做：

$$Y = XW + b$$

Batch 的本质不是“数学规则变了”

Batch 只是把许多个“单样本 $xW + b$ ”打包成一个更大的矩阵运算。每一行仍然在做相同的 Linear Mapping，只是一起算。

4 二维 Batch: $(B, D) @ (D, O) \rightarrow (B, O)$

设：

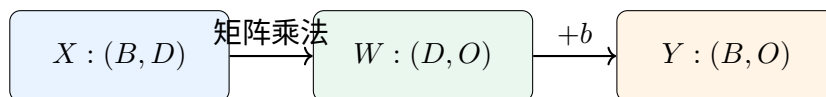
$$X : (3, 4), \quad W : (4, 2), \quad b : (2,)$$

那么：

$$XW : (3, 2)$$

再加 Bias 后：

$$Y : (3, 2)$$



4.1 每一行究竟发生了什么？

如果：

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

则：

$$XW + b = \begin{bmatrix} x_1W + b \\ x_2W + b \\ x_3W + b \end{bmatrix}$$

所以 Batch 运算没有创造“第四种神秘计算”。它仍然是每个样本独立使用同一套参数。

常见误区：每个样本是不是有自己的 Weight?

不是。若一个 Linear 层只有一套 W, b , Batch 中的所有样本都共享它们。否则模型参数会随着 Batch Size 改变，这是完全不同的模型定义。

5 从 (B, D) 进入 LLM：为什么多出一个 T ？

普通表格数据通常一个样本就是一个向量。但语言数据不同：一段文本包含一串 Token。

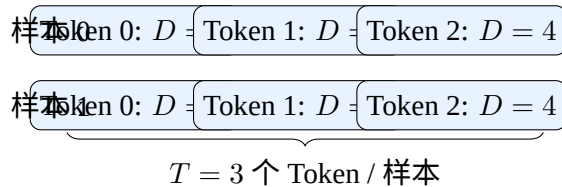
假设：

- 一次处理 2 条句子： $B = 2$ ；
- 每条句子固定 3 个 Token： $T = 3$ ；
- 每个 Token 用 4 个数字表示： $D = 4$ 。

那么输入 Shape 是：

$$X : (2, 3, 4) = (B, T, D)$$

$B = 2$ ：两个样本



三个轴的语义

对典型 LLM hidden states：

$$(B, T, D)$$

可以先用一句话读出来：“**B 条序列，每条有 T 个 Token，每个 Token 是 D 维向量。**”

6 关键问题：Linear 怎样作用于三维 Tensor？

假设：

$$X : (B, T, D)$$

现在有一个：

$$W : (D, O), \quad b : (O,)$$

那么：

$$Y = XW + b$$

得到：

$$Y : (B, T, O)$$

6.1 内部到底发生了什么？

它等价于：对每一个 (b, t) 位置上的 Token 向量：

$$X[b, t, :] : (D,)$$

都执行同一套：

$$X[b, t, :]W + b$$

得到：

$$Y[b, t, :] : (O,)$$

一个极小例子

输入：

$$X : (2, 3, 4)$$

表示 2 个样本、每个 3 个 Token、每个 Token 4 维。

若 Linear 是 $4 \rightarrow 2$ ：

$$W : (4, 2)$$

那么输出：

$$Y : (2, 3, 2)$$

前面的 $B = 2, T = 3$ 被保留；最后一个特征维从 4 变成 2。

7 一个极其重要的判断：Linear 本身不会混合 Token

这是初学 Transformer 时非常容易误解的地方。

对：

$$X : (B, T, D)$$

执行一个普通 Linear：

$$Y = XW + b$$

时，每个 Token 位置单独使用同一套 W, b 。

也就是说：

$$Y[b, t, :]$$

只直接来自：

$$X[b, t, :]$$

而不会因为这个 Linear 层就自动读取 $X[b, t - 1, :]$ 或 $X[b, t + 1, :]$ 。

那 Transformer 里不同 Token 什么时候互相“看见”？

关键发生在 **Attention**。Q/K/V 的 Linear projection 本身主要是逐 Token 投影；真正把不同 Token 之间建立依赖关系的，是后续 QK^T 与加权求和。

因此以后你可以把 Transformer 拆成两类操作：

1. **逐 Token 的特征变换**：Linear、部分 MLP/FFN 等；
2. **Token 之间的信息交互**：Attention 等。

8 为什么三维 Linear 等价于先展平 $B \times T$?

有：

$$X : (B, T, D)$$

可以把前两个轴临时合并：

$$X_{flat} : (B \times T, D)$$

然后：

$$Y_{flat} = X_{flat}W + b$$

得到：

$$Y_{flat} : (B \times T, O)$$

最后恢复：

$$Y : (B, T, O)$$

只要 reshape 时没有打乱元素顺序，这与直接对三维 Tensor 做 Linear 是等价的。

核心概念

理解这个等价关系非常重要，因为它说明：**所谓“3D Tensor 做 Linear”并没有新数学，本质仍然是很多个 D 维向量共享同一套映射。**

9 Bias 在三维 Tensor 中如何 Broadcasting?

若：

$$Z : (B, T, O)$$

而：

$$b : (O,)$$

那么：

$$Y = Z + b$$

时，Bias 会被加到每一个 Batch、每一个 Token 的最后一个输出维上。

直觉上相当于：

$$b \rightarrow (1, 1, O) \rightarrow (B, T, O)$$

但通常不需要真的把 Bias 复制很多份，框架通过 Broadcasting 规则完成。

Broadcasting 既方便又危险

很多 Shape Bug 不会立刻报错，因为两个 Shape “碰巧可以广播”。科研代码调试时，不能只问“程序有没有报错”，还要问“它广播的轴是不是我想要的轴”。

10 Batch Size、Sequence Length 与参数量的关系

假设一个 Linear：

$$D \rightarrow O$$

参数量是：

$$D \times O + O$$

例如 $4 \rightarrow 2$ ：

$$4 \times 2 + 2 = 10$$

无论：

$$B = 2, T = 3$$

还是：

$$B = 128, T = 4096$$

只要 D, O 不变，这一层参数量仍然是 10。

参数量和计算量不要混淆

参数量主要由模型结构决定；计算量/激活显存会明显受到 Batch Size、Sequence Length 等输入 Shape 的影响。

10.1 为什么 B 和 T 大了会更贵？

因为需要处理的 Token 数大约是：

$$B \times T$$

对于逐 Token Linear，计算工作量会随 Token 数增加。

Attention 更特殊：标准 Self-Attention 中还会出现与 T^2 相关的矩阵，因此 Sequence Length 对计算和显存尤其敏感。这个细节以后正式学 Attention 时再推导。

11 Mini-batch：真实训练中常用的折中

如果训练集有 100 万条数据，不太可能一次全部放入显存。因此通常把数据拆成很多小批次。

例如：

$$\text{Dataset Size} = 10000, \quad \text{Batch Size} = 100$$

那么一轮完整遍历大约需要：

$$10000/100 = 100 \text{ 个 Batch}$$

先建立直觉，后面优化器章节再深入

Mini-batch 的作用不仅是省显存。它还会影响每次估计出来的 Gradient 的噪声程度，从而影响训练动态。第 34–35 讲会专门比较 Full Batch、SGD 与 Mini-batch SGD，并正式定义 Step / Epoch。

12 NumPy：直接验证 $(B, T, D) \rightarrow (B, T, O)$

下面的代码就是本讲实验的核心：

```
import numpy as np

X = np.arange(2 * 3 * 4, dtype=float).reshape(2, 3, 4)
W = np.ones((4, 2))
b = np.zeros(2)

Y = X @ W + b

print(X.shape)  # (2, 3, 4)
print(W.shape)  # (4, 2)
print(Y.shape)  # (2, 3, 2)
```

12.1 怎样读这一段代码？

不要只记输出。应该逐步问：

1. X 的最后一维是 4；
2. W 的第一维也是 4，因此可以做矩阵乘法；
3. W 把最后一维 4 变成 2；
4. 前面的 $(2, 3)$ 不参与这个特征映射，因此保留；
5. 结果是 $(2, 3, 2)$ 。

13 PyTorch：nn.Linear 为什么能直接处理三维 Tensor？

PyTorch 中：

```
import torch
import torch.nn as nn

layer = nn.Linear(4, 2)
X = torch.randn(2, 3, 4)
Y = layer(X)
```

```
print(Y.shape) # torch.Size([2, 3, 2])
```

`nn.Linear(in_features, out_features)` 的规则可以先记成：

PyTorch Linear 的 Shape 规则

只要输入最后一维是 `in_features`，前面可以有任意多个轴：

$$(*, D_{in}) \rightarrow (*, D_{out})$$

其中 `*` 表示“前面那些轴保持原样”。

因此：

$$(B, T, D) \rightarrow (B, T, O)$$

只是这一通用规则的一个具体情况。

14 与 Transformer 的第一次完整 Shape 对接

假设 hidden states：

$$X : (B, T, D_{model})$$

Attention 前需要产生 Q、K、V：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

如果暂时让每个投影都保持相同 hidden size：

$$W_Q, W_K, W_V : (D_{model}, D_{model})$$

则：

$$Q, K, V : (B, T, D_{model})$$

与 LLM / Transformer / Agent Systems 的连接

到这里你应该已经能看见：Q/K/V 的生成阶段并不神秘，它就是本讲的三维 Batch Linear。后面真正新鲜的部分，是把最后一维拆成多个 Head，再通过 QK^T 让不同 Token 交互。

14.1 FFN / MLP 也是同样的 Shape 思路

例如：

$$(B, T, 768) \rightarrow (B, T, 3072) \rightarrow (B, T, 768)$$

它并不是把 3D Tensor “变成一个特殊对象”。而是：每个 Token 的 768 维表示先被映射到 3072 维，再映射回来，前面的 B, T 保留。

15 与 Agent Systems 有什么关系？

Agent Systems 研究常常把基础模型当作 API，看不到内部 Batch。但只要你做以下任务，Batch/Shape 就会重新出现：

- 本地模型推理与批量评测；
- 多 Agent Benchmark 一次跑很多任务；
- 对多个轨迹做 Embedding；
- 读模型内部激活或 hidden states；
- 做 LoRA / 微调 / probe；
- 分析模型计算成本与显存；
- 阅读 Transformer 结构或训练实现。

科研阅读中的一个实用习惯

看到一个 Tensor，不要急着看它的数值。先写出：

Shape + 每个轴的语义

例如：

$(8, 512, 4096) = (8 \text{ 个样本}, 512 \text{ 个 Token}, 4096 \text{ 维 hidden state})$

很多论文代码只要把 Shape 链追通，结构就已经理解了一半。

16 常见误区

误区 1：Batch 越大，模型参数越多

错误。Batch Size 影响一次喂多少数据、显存、计算吞吐和优化动态，但普通 Linear 的参数数仍由 D_{in}, D_{out} 决定。

误区 2：(B, T, D) 做 Linear 会让 Token 相互交流

错误。普通 Linear 主要作用在最后的特征维。不同 Token 的交互需要 Attention、卷积、RNN 状态传播等机制。

误区 3: Batch Dimension 永远必须放第一维

不是数学定律，只是常见约定。不同库/任务可能有不同布局，例如某些旧 RNN API 使用 (T, B, D) 。一定以具体文档与代码为准。

误区 4: reshape 后 Shape 对了，语义就一定对

不一定。Shape 只是尺寸，不能保证轴语义正确。把 Batch 和 Token 顺序弄乱，即使元素总数一样、代码不报错，结果也可能完全错误。

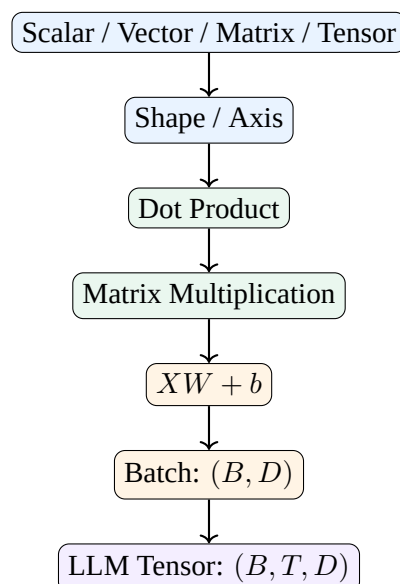
17 本讲实验：从二维 Batch 到三维 Token Batch

实验文件：lesson06_batch_tensor_experiment.py

实验采用“预测 Shape → 执行 → 对照 → 解释”的方式，包括：

1. 二维 Batch 与逐样本循环对照；
2. 构造 $(B, T, D) = (2, 3, 4)$ 的微型 LLM Tensor；
3. 直接 3D Linear 与 flatten $(B \times T, D)$ 的等价性；
4. 逐 Token 循环与向量化计算对照；
5. Bias Broadcasting；
6. Batch/Sequence Length 与参数量的关系；
7. PyTorch `nn.Linear` 对三维 Tensor 的验证。

18 本讲知识地图：第一阶段到这里闭环



第一阶段总验收

如果你看到：

$$X : (32, 128, 768), \quad W : (768, 3072)$$

应该能够立刻判断：

$$XW : (32, 128, 3072)$$

并解释：32 是 Batch，128 是 Token 数，768 是每个 Token 的输入特征维，3072 是输出特征维；所有 Token 共享同一套 W 。

19 练习题

1. $X : (64, 128)$ ， $W : (128, 256)$ ，输出 Shape 是什么？
2. $X : (8, 512, 768)$ ，Linear 为 $768 \rightarrow 3072$ ，输出 Shape 是什么？
3. 上题的 Batch Size 改成 16，这一 Linear 的参数数量是否改变？为什么？
4. 若 $X : (2, 3, 4)$ ，为什么把它 reshape 成 $(6, 4)$ 后可以使用同一个 $4 \rightarrow 2$ Linear？
5. 对 (B, T, D) 做 Linear，是否意味着第 3 个 Token 的输出直接使用了第 2 个 Token 的输入？
6. Bias 为 $(O,)$ 时，为什么能加到 (B, T, O) ？
7. hidden states 为 $(4, 100, 512)$ ， $W_Q : (512, 512)$ ，Q 的 Shape 是什么？
8. 为什么 Batch Size 增大通常会增加显存，但不直接增加模型参数量？

20 答案

1. $(64, 256)$ 。
2. $(8, 512, 3072)$ 。
3. 不改变。参数量只由 $768 \times 3072 + 3072$ 决定，与 B 无关。
4. 因为合并 B, T 后只是把 6 个 Token 当成 6 个独立的 4 维向量；每个仍使用同一套 Weight。
5. 不意味着。普通 Linear 只对该位置最后一维做映射；Token 间交互要靠其他机制，例如 Attention。
6. Broadcasting 会把 $(O,)$ 视为对每一个 (b, t) 位置的输出向量加同一组 Bias。
7. $(4, 100, 512)$ 。
8. 更大的 Batch 需要保存/计算更多样本的中间激活，因此显存增加；参数本身那套 W, b 没有复制成更多可学习参数。

21 下一讲预告：从“线性计算”进入“神经元”

第一阶段到这里结束。我们已经知道数据如何组织、Linear 如何计算，但还没有真正解释：

为什么只堆很多 Linear 层还不够？神经网络究竟靠什么获得非线性表达能力？

第 07 讲将正式进入第二阶段：**神经元究竟是什么**。我们会从一个加权和出发，为后面的 Activation Function 和 MLP 做准备。